HOJA DE RUTA DE ESTUDIO

Claude Certified Architect - Foundations (CCAR-F)

Guía práctica en español basada en el Exam Guide v1.0 adjunto, revisión del repositorio y recursos públicos verificados.

Objetivo: llegar al examen con criterio arquitectónico, proyectos demostrables y una estrategia de práctica de bajo costo.

EXAMEN	QUÉ SIGNIFICA PARA ESTUDIAR
60 preguntas · 120 minutos	4 escenarios seleccionados de un banco de 6; opción múltiple y selección múltiple. Cada pregunta indica cuántas respuestas elegir.
720 / 1.000 para aprobar	Es una puntuación escalada. No debe interpretarse como una garantía literal de que basta responder correctamente al 72% en cualquier formulario.
Enfoque hands-on	La guía recomienda experiencia práctica con Claude API, Claude Code, Agent SDK y MCP, no memorizar definiciones aisladas.

Cómo usar este documento

Sigue el plan de 6 semanas si dispones de unas 2 horas diarias, 5 días por semana (aprox. 60 horas). Si tienes menos tiempo, conserva el orden, pero no omitas las pruebas prácticas ni la revisión de errores. La secuencia y las horas son una recomendación de planificación, no un requisito oficial.

Fuente primaria: Exam Guide v1.0, efectivo desde julio de 2026, adjunto por el usuario. Las recomendaciones prácticas y la estrategia de modelos son síntesis y criterio de estudio; los recursos externos se etiquetan por su procedencia.

1. Prioriza según el blueprint oficial

La guía asigna pesos aproximados; usa estos porcentajes para repartir el tiempo y no estudies cada tema por igual.

DOMINIO	PESO	CONTENIDO QUE DEBES DOMINAR	HORAS SUGERIDAS
1. Arquitectura y orquestación de agentes	**27%**	Agentic loop y stop_reason; coordinator/subagents; delegación paralela; hooks; descomposición; sesiones, resume y fork.	**16 h**
2. Diseño de herramientas e integración MCP	**18%**	Descripciones precisas; tools/resources/prompts; errores estructurados; tool_choice; permisos acotados; .mcp.json.	**11 h**
3. Claude Code: configuración y workflows	**20%**	Jerarquía CLAUDE.md; rules con glob; skills/commands; plan mode vs ejecución directa; iteración y CI/CD.	**12 h**
4. Prompt engineering y salida estructurada	**20%**	Criterios explícitos, few-shot, tool_use + JSON Schema, validación semántica, retries, batches y revisión independiente.	**12 h**
5. Contexto y confiabilidad	**15%**	Case facts, reducción de output, escalamiento, propagación de errores, scratchpads, recuperación, confianza calibrada y procedencia.	**9 h**

Regla de repaso: cuando una opción parece correcta porque “el prompt lo pide”, pregunta si el requisito necesita una garantía determinista. Para identidad, dinero, permisos o reglas obligatorias, prioriza controles de programa/hooks/gates frente a instrucciones probabilísticas.

Señales que se repiten en las preguntas de ejemplo

• No confundas una llamada de herramienta con la finalización del turno: continúa cuando stop_reason = tool_use y termina cuando stop_reason = end_turn.

• Un subagente no hereda automáticamente la conversación del coordinador. Pásale explícitamente hallazgos, límites y metadatos de procedencia.

• No ocultes fallos devolviendo un resultado vacío como si fuera éxito: distingue error transitorio, validación, permiso, regla de negocio y consulta válida sin coincidencias.

• Usa mensajes de error concretos y campos como errorCategory, isRetryable, partialResults y attemptedQuery.

• Las reglas obligatorias deben aplicarse programáticamente; para mejorar selección de herramientas, empieza por descripciones con ejemplos, entradas, límites y casos de uso diferenciados.

• Para salida estructurada, JSON Schema evita errores de sintaxis, pero no garantiza que los datos sean semánticamente correctos: valida cálculos, relaciones y consistencia.

2. Plan de seis semanas

Ritmo propuesto: 2 horas al día, 5 días por semana. En cada sesión: 20 min de lectura, 70 min de implementación, 20 min de pruebas y 10 min de notas. Usa un repositorio de práctica con commits pequeños y un README por ejercicio.

Distribuye el trabajo por etapas; si un bloque termina antes, usa el tiempo restante para escribir pruebas y analizar errores, no para saltar al siguiente sin validar.

Semana 1 - Agentic loop y herramientas (10 h)

☐ Implementa un bucle Claude API con 2 herramientas simples, manejo de tool_use/end_turn, historial de mensajes y límite defensivo de seguridad (no como sustituto del stop_reason).

☐ Diseña contratos para get_customer, lookup_order y process_refund; agrega descripción, ejemplos válidos, casos límite y errores categorizados.

☐ Prueba timeout, permiso denegado, entrada inválida, regla de negocio y resultado válido sin coincidencias. Registra qué debe reintentarse y qué debe escalarse.

☐ Entrega: pruebas unitarias del loop y matriz de selección de herramientas con 15 consultas ambiguas.

Semana 2 - MCP y permisos de herramientas (10 h)

☐ Crea un servidor MCP de tickets/documentos con herramientas de consulta/acción, un resource de catálogo y al menos un prompt reutilizable.

☐ Configura un cliente MCP; confirma el descubrimiento de tools/resources. Diferencia tools (acciones del modelo), resources (datos expuestos) y prompts (plantillas iniciadas por el usuario).

☐ En Claude Code configura un MCP compartido en .mcp.json con variable de entorno para secreto y uno personal en configuración de usuario.

☐ Entrega: README de contratos, ejemplo de respuestas de error, y tabla de herramientas por rol con privilegio mínimo.

Semana 3 - Orquestación multiagente (10 h)

☐ Crea coordinator + dos subagentes: búsqueda y análisis de documentos; después incorpora síntesis. Inyecta explícitamente las entradas necesarias a cada subagente.

☐ Paraleliza subtareas en una misma fase, evita duplicación y agrega un ciclo de refinamiento si faltan áreas del tema.

☐ Simula timeout de un subagente; el coordinador debe decidir reintento, alternativa o reporte parcial con lagunas de cobertura.

☐ Entrega: informe con afirmación, extracto de evidencia, URL/documento, fecha y estado de cobertura; prueba una contradicción entre fuentes.

Semana 4 - Claude Code como workflow de equipo (10 h)

☐ Organiza CLAUDE.md raíz con estándares comunes; .claude/rules/ con rutas glob para APIs y tests; comandos compartidos en .claude/commands/ y skills con SKILL.md.

☐ Compara plan mode en una migración multiarchivo y ejecución directa en un cambio de una sola función.

☐ Crea una skill exploratoria con context: fork y allowed-tools restringidos. Verifica qué instrucciones se cargan y evita un CLAUDE.md monolítico.

☐ Entrega: repositorio preparado para un equipo, con instrucciones de build/test, reglas por rutas y una checklist de PR.

Semana 5 - Extracción estructurada y CI/CD (10 h)

☐ Implementa extracción de documentos a JSON Schema: requeridos, opcionales/nullable, enums y other + detail. Si falta información, devuelve null/unclear, no inventes.

☐ Agrega validación semántica (sumas, fechas, consistencia) y retry con el error concreto; no reintentes cuando el dato no existe en la fuente.

☐ Construye un workflow CI que revise cambios con una instancia separada de la que generó el código. Usa salida JSON validada para comentarios del PR.

☐ Entrega: muestras de 20 documentos, informe por campo/tipo de documento y política de revisión humana.

Semana 6 - Contexto, simulacro y corrección (10 h)

☐ Persiste case facts y scratchpad/manifiesto de estado; trunca resultados de herramientas a campos relevantes; documenta reglas para resumir sin perder montos, fechas o IDs.

☐ Practica resume vs sesión nueva con resumen estructurado vs fork para probar enfoques alternativos. Si los archivos cambiaron, informa qué resultados anteriores están obsoletos.

☐ Resuelve las 12 preguntas de ejemplo del Exam Guide explicando por qué cada distractor falla. Haz un simulacro propio de 60 preguntas/120 minutos.

☐ Analiza errores por dominio. Repite primero los dominios bajo 80% en práctica; este 80% es una meta interna sugerida, no el puntaje oficial.

Si vas justo de tiempo

No sacrifiques las semanas 1, 3, 4 ni 5. Los proyectos combinados cubren varios dominios. Baja el tiempo dedicado a adornar la interfaz: la evidencia de aprendizaje son contratos, tests, trazabilidad de decisiones y fallos recuperables.

3. Proyectos prácticos que cubren el examen

Recomendación: usa un solo caso de negocio -por ejemplo, soporte de pedidos y extracción de facturas- para construir piezas acumulativas. Puedes implementar con Python o TypeScript; la guía no examina un lenguaje específico.

Proyecto A - Agente de soporte con escalamiento

Base de Agentic loop y Domain 1/2/5. Herramientas: get_customer, lookup_order, process_refund, escalate_to_human. Verificación obligatoria de identidad antes de operar; bloquear reembolsos mayores a un umbral por código/hook; resumir contexto para el agente humano.

Prueba de salida: Criterios de aceptación: stop_reason correcto; 5 clases de error; pruebas de intento de salto de verificación; escalamiento por petición explícita o falta de política; nada de acciones financieras si no hay cliente verificado.

Proyecto B - MCP para incidencias y documentación

Enfocado en Domain 2/3. MCP server con herramientas search_ticket, create_ticket y get_runbook; resource de catálogo de runbooks; errores estructurados y permisos distintos para lector/escritor. Conecta el servidor desde Claude Code usando configuración de proyecto y secretos vía variables de entorno.

Prueba de salida: Criterios de aceptación: tool descriptions que resuelven nombres parecidos; resource reduce llamadas de exploración; errores transitorios distinguibles de “no hay resultados”; ninguna credencial en Git.

Proyecto C - Pipeline multiagente de investigación

Enfocado en Domain 1/2/5. Coordinator delega a búsqueda y análisis de documentos en paralelo y pasa hallazgos estructurados a síntesis. Añade verificación simple con un tool acotado y eleva investigaciones complejas al coordinador.

Prueba de salida: Criterios de aceptación: separación de tareas y sin duplicación; estado explícito de cobertura; fallos parciales recuperables; fuentes/fechas junto a afirmaciones; conflictos presentados sin inventar una resolución.

Proyecto D - Extractor JSON para documentos

Enfocado en Domain 4/5. Extrae proveedor, número, fecha, partidas, total, moneda y campos opcionales desde facturas sintéticas. Valida JSON Schema y reglas semánticas, ejecuta un retry con el error concreto y deriva baja confianza a revisión humana.

Prueba de salida: Criterios de aceptación: documento con campos ausentes no produce datos inventados; valida total vs partidas; evalúa precisión por tipo de documento/campo; reintenta solo errores corregibles.

Proyecto E - Revisor de pull requests con Claude Code

Enfocado en Domain 3/4/5. Workflow CI con claude -p, salida estructurada y criterios explícitos. El revisor es independiente del agente generador; análisis por archivo y pasada adicional de integración para data flow entre archivos.

Prueba de salida: Criterios de aceptación: sin espera interactiva; salida parseable por máquina; informes sin duplicados tras nuevos commits; falsos positivos rastreables con detected_pattern; se prueban criterios contra ejemplos positivos y negativos.

Estructura recomendada del repositorio

```
/docs/architecture.md
/docs/decisions/ADR-001-...md
/src/agent/loop.py
/src/tools/ (o mcp_server/)
/src/agents/coordinator.py
/src/agents/researcher.py
/src/extraction/schema.json
/tests/unit/ y /tests/scenarios/
/evals/dataset.jsonl
/scratchpad/README.md
CLAUDE.md
.claude/rules/ y .claude/skills/
```

No hace falta que todos los proyectos sean servicios desplegados en la nube. El Exam Guide excluye despliegue de MCP, configuración de proveedores cloud, OAuth y cálculo detallado de precios API; enfócate en interfaces, flujo de control, contexto, errores y calidad.

4. Recursos de estudio y cómo usarlos

Prioriza primero la guía adjunta y los materiales oficiales. Los recursos comunitarios son complementarios y pueden contener preguntas o interpretaciones no validadas por Anthropic.

A. Material oficial y gratuito

Anthropic Partner Academy - página oficial CCAR-F y dominios
https://anthropic-partners.skilljar.com/claude-certified-architect-foundations-certification

Building with the Claude API - curso oficial (API, tool use, structured output, agentes, MCP)
https://academy.claude.com/es/courses/building-with-the-claude-api

Introduction to Model Context Protocol - curso oficial y ejercicio de servidor/cliente MCP
https://academy.claude.com/courses/introduction-to-model-context-protocol

Claude Code in Action - curso oficial gratuito
https://anthropic.skilljar.com/claude-code-in-action

Claude Academy - colección Build with Claude
https://academy.claude.com/collections/build-with-claude

Documentación oficial de SDKs para Claude API
https://platform.claude.com/docs/en/cli-sdks-libraries/overview

Documentación oficial de Agent Skills
https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

Mejores prácticas de Claude Code (Anthropic)
https://www.anthropic.com/research/claude-code-best-practices

Ejemplos oficiales MCP quickstart-resources
https://github.com/modelcontextprotocol/quickstart-resources

B. Repositorios y guías comunitarias (no oficiales)

Tu fork de materiales de estudio - franklinrony/claude-certified-architect
https://github.com/franklinrony/claude-certified-architect

Ravn Edition - guía de estudio en español/inglés/portugués y checklist
https://ravnhq.github.io/claude-certified-architect/

Repositorio Ravn: guías, ejercicios y notas de recursos
https://github.com/ravnhq/claude-certified-architect

Guía práctica adicional con explicación de dominios y ejercicios
https://github.com/dnacenta/claude-certified-architect

Guía CCAR-F con proyectos y banco de práctica (tercero)
https://github.com/saqif-i/ccar-f-study-guide

Cómo estudiar cada recurso

• Lee el objetivo oficial de cada tarea antes de ver una explicación. Implementa el patrón sin copiarlo entero.

• Resuelve primero las preguntas de ejemplo del PDF oficial sin mirar respuesta. Para cada distractor, escribe por qué no soluciona la causa raíz.

• Usa bancos comunitarios para practicar decisiones, no como predicción de preguntas reales. La guía oficial y la documentación primaria prevalecen ante discrepancias.

• Mantén un registro de errores: pregunta/escenario, decisión elegida, señal decisiva ignorada, regla correcta, prueba que lo demuestra. Revisa ese registro cada semana.

5. Modelos y presupuesto: cómo practicar gastando poco

En la guía oficial del examen, la elección de modelo no es el centro del blueprint y se excluyen cálculos detallados de precios API. Esta sección es una recomendación para ejecutar proyectos personales con un costo controlado; los precios cambian y conviene confirmar la tabla de Anthropic antes de activar tráfico real.

Política recomendada: una sola variable cada vez

• Empieza sin API: diseña contratos JSON, schemas, tests, reglas de selección y fixtures; los tests de control de flujo y validación se pueden hacer con respuestas simuladas.

• Para las primeras iteraciones de clasificación, extracción simple, normalización y formato de salida, prueba un modelo económico disponible en tu cuenta (por ejemplo, familia Haiku) y limita max_tokens.

• Usa un modelo intermedio (por ejemplo, Sonnet) cuando necesites mejor razonamiento de varios pasos, depuración compleja o síntesis multiagente. Reserva el modelo más caro para casos donde una evaluación comparativa demuestre una ventaja real.

• No ejecutes todos los subagentes con el modelo caro por defecto. Configura el modelo por rol: económico para clasificación y verificación simple; intermedio para coordinación, síntesis y tareas ambiguas.

• Para evaluaciones nocturnas no interactivas, compara Message Batches API: la guía oficial indica descuento del 50%, pero hasta 24 horas y sin SLA de latencia; no es adecuado para bloqueos de pre-merge.

• Usa una muestra fija de 20-30 casos, registra exactitud por campo, falsos positivos, fallos de schema y costo estimado antes de ampliar el lote. No confundas salida JSON válida con extracción correcta.

• Mantén prompts cortos: devuelve campos útiles desde las herramientas, conserva hechos críticos en case facts y no reinyectes logs o documentos completos si no hacen falta.

• Evita que el agente se autoevalúe como única métrica de confianza. Usa un dataset etiquetado, revisión humana estratificada y resultados por tipo de documento/campo.

Precios públicos encontrados en documentación de Anthropic

La página de precios consultada muestra para los modelos visibles, a la fecha de consulta: Haiku 4.5 a USD 1 por millón de tokens de entrada y USD 5 por millón de salida; Sonnet 4.5 a USD 3/15; Opus 4.5 a USD 5/25. La misma página indica descuento del 50% para batches. Puede haber modelos y tarifas más nuevos según la cuenta/región; usa la página viva como fuente de verdad antes de presupuestar.

Tabla oficial de precios API y Message Batches
https://platform.claude.com/docs/en/about-claude/pricing

Ejemplo de control de gasto

Define un presupuesto máximo para el ejercicio (por ejemplo, USD 3-5 como límite personal, no una predicción de costo). Guarda los tokens de entrada/salida por ejecución; agrega una bandera local que impida repetir una llamada si la misma combinación de input, modelo y versión de prompt ya está en caché de resultados de pruebas. Ejecuta evaluaciones completas sólo tras aprobar un subconjunto pequeño.

¿Claude Code o API?

Para aprender archivos CLAUDE.md, rules, skills, plan mode, herramientas integradas y workflows de equipo, Claude Code es el medio natural, pero el costo depende del plan/licencia que uses. Para medir tool_choice, stop_reason, esquemas y batch, usa la API. No asumas que una suscripción de chat incluye créditos API: revisa por separado el plan y la facturación que aplican a tu cuenta.

6. Lista de chequeo antes del examen

No programes el examen sólo porque terminaste los cursos. Usa esta checklist basada en las capacidades que la guía oficial pide practicar.

☐ Puedo explicar e implementar el agentic loop y terminar por stop_reason, no por buscar una frase en texto.

☐ Puedo hacer que el coordinador delegue selectivamente, pase contexto explícito y refine la investigación si detecta lagunas.

☐ Puedo explicar cuándo hace falta un gate determinista, un hook o una instrucción de prompt.

☐ Puedo diseñar descripciones de tools que no se confundan y errores con categoría, reintentabilidad y resultados parciales.

☐ Puedo diferenciar tools, resources y prompts MCP; configurar scope de proyecto/usuario y no commitear secretos.

☐ Puedo elegir correctamente Read, Write, Edit, Bash, Grep y Glob para cada tarea de exploración o modificación.

☐ Puedo organizar CLAUDE.md, .claude/rules/, commands y skills; diferenciar plan mode de ejecución directa.

☐ Puedo correr Claude Code sin interacción en CI y devolver hallazgos estructurados para integrarlos en un PR.

☐ Puedo crear JSON Schema con nullable/optional y validar errores semánticos aunque el JSON sea válido.

☐ Puedo explicar cuándo conviene retry, cuándo no ayuda y cuándo mandar a revisión humana.

☐ Puedo recortar salidas de herramientas, persistir case facts/scratchpad y resumir sin perder IDs, cantidades y fechas.

☐ Puedo preservar fuentes, fechas, valores contradictorios y lagunas de cobertura en el informe final.

☐ He repasado las 12 preguntas de muestra del Exam Guide y puedo explicar todos los distractores.

☐ He completado un simulacro de 60 preguntas en 120 minutos y revisado el resultado por dominio.

Estrategia durante el examen

• Tiempo: 120 minutos para 60 preguntas da un promedio de 2 minutos por pregunta. No te atasques en una sola; marca mentalmente el supuesto que cambia la decisión y avanza.

• En preguntas de respuesta múltiple, cuenta las respuestas requeridas antes de seleccionar. Evita escoger opciones sólo porque “también podrían funcionar”; busca la solución más directa a la causa raíz descrita.

• Lee la condición de latencia, seguridad, escalamiento y ámbito del contexto. Muchas alternativas son plausibles, pero sólo una responde al problema concreto sin sobreingeniería.

Referencia del examen y nota sobre versiones

El PDF de origen declara versión 1.0, efectiva en julio de 2026, código CCAR-F. Sus datos logísticos y pesos están reproducidos aquí para el plan de estudio. Revisa la página oficial de certificación justo antes de reservar, porque guía, políticas, disponibilidad y precios pueden cambiar.

Ficha oficial de certificación y registro
https://anthropic-partners.skilljar.com/claude-certified-architect-foundations-certification

Base documental

1) Claude Certified Architect - Foundations Exam Guide v1.0, archivo adjunto de 39 páginas: blueprint, escenarios, objetivos, ejercicios, preguntas de muestra y exclusiones. 2) Repositorio GitHub indicado por el usuario: se comprobó que es público y que es un fork del repositorio paullarionov/claude-certified-architect; por limitaciones de lectura de archivos del conector no se pudo inspeccionar su árbol interno completo. 3) Recursos de Anthropic Academy y documentación de precios enlazados en esta hoja de ruta. Los ejercicios propuestos son diseño propio basados en los objetivos oficiales, no proyectos oficiales ni preguntas reales del examen.